

Programming Assignment #4

Make realistic and nice-looking images using ray tracing

Due:

Please check on ETL

Submission:

Please zip all files (program + supplemental material) then upload it on the ETL

Specification:

- Language & Recommended APIs
 - Language
 - Python
 - Image processing
 - Pillow: <https://pillow.readthedocs.io/en/stable/handbook/tutorial.html>
 - Multiprocessing
 -  Python documentation [multiprocessing — Process-based parallelism](#)
 -  PyTorch [PyTorch](#)

- Cornell box
 - Set up your own Cornell box with which implement ray-tracing
 - When setting up the Cornell box, use any shapes below
 - Sphere
 - Box
 - Cone
 - Cylinder
 - Capsule
 - Plane
 - Mesh models which satisfy any condition below
 - Ones you created in HW1 or HW2
 - Any free-model allowed in HW3
 - Approximate the area light source as a collection of point lights
 - Your submitted images should be larger than **200px X 200px**
 - **[2pt]** if shadow is correctly implemented
 - **[2pt]** if reflection is correctly implemented
 - **[2pt]** if refraction is correctly implemented
 - **[1pt]** Submit a comparison on how your scene can be improved according to the number of recursion depth at least three different depths (1, 2, and 4).
 - **[1pt]** Try to use more than one ray per pixel. Then, report how much image quality has improved. In case where the quality improvement is clearly visible, you will get a point.

- Artistic score
 - Submit 3 best screenshots or a video (less than 1min) on your scene.
 - The scores will be evaluated by all participants. Initially, TAs and the lecturer will select the top 10 results. From these selected results, the students will then vote for their 2-3 favorite choices.
 - **[2pt]** 1st-3rd place
 - **[1pt]** 4rd-6th place
 - If necessary, implement any ray-object intersection acceleration techniques (e.g., BVH, K-D Tree), which were discussed in the lecture. If you setup your scene with only simple primitives (such as spheres or box), you might not need this. But if you want to bring some interesting geometries into your scene to make it interesting and beautiful, then you would probably need this.
 - It is also OK to use any other acceleration techniques (e.g., GPU/CPU parallelization, or PyTorch).
- **[2pt]** Supplemental material
 - The document should include (can be written either in Korean or in English):
 - Describe what you implemented, and what you haven't
 - A detailed instruction on how to run and use your program
 - Screenshots of your results for each problem, more specifically:
 - The Cornell box image you rendered
 - A comparison on the number of recursion depth
 - A comparison on the number of rays per pixel.
 - Images or a video for voting
 - Any reference you referred

Cornell Box Example Images:

